01. data types

Julia Gallucci HB.Sc, M.Sc, Ph.D.

Campbell Family Mental Health Research Institute, CAMH

Institute of Medical Science | University of Toronto

Learning objectives

- Introduction to Jupyter notebooks
- Understand core python data types
- Differentiate between statements and expressions
- Use logical operators
- Create and use variables

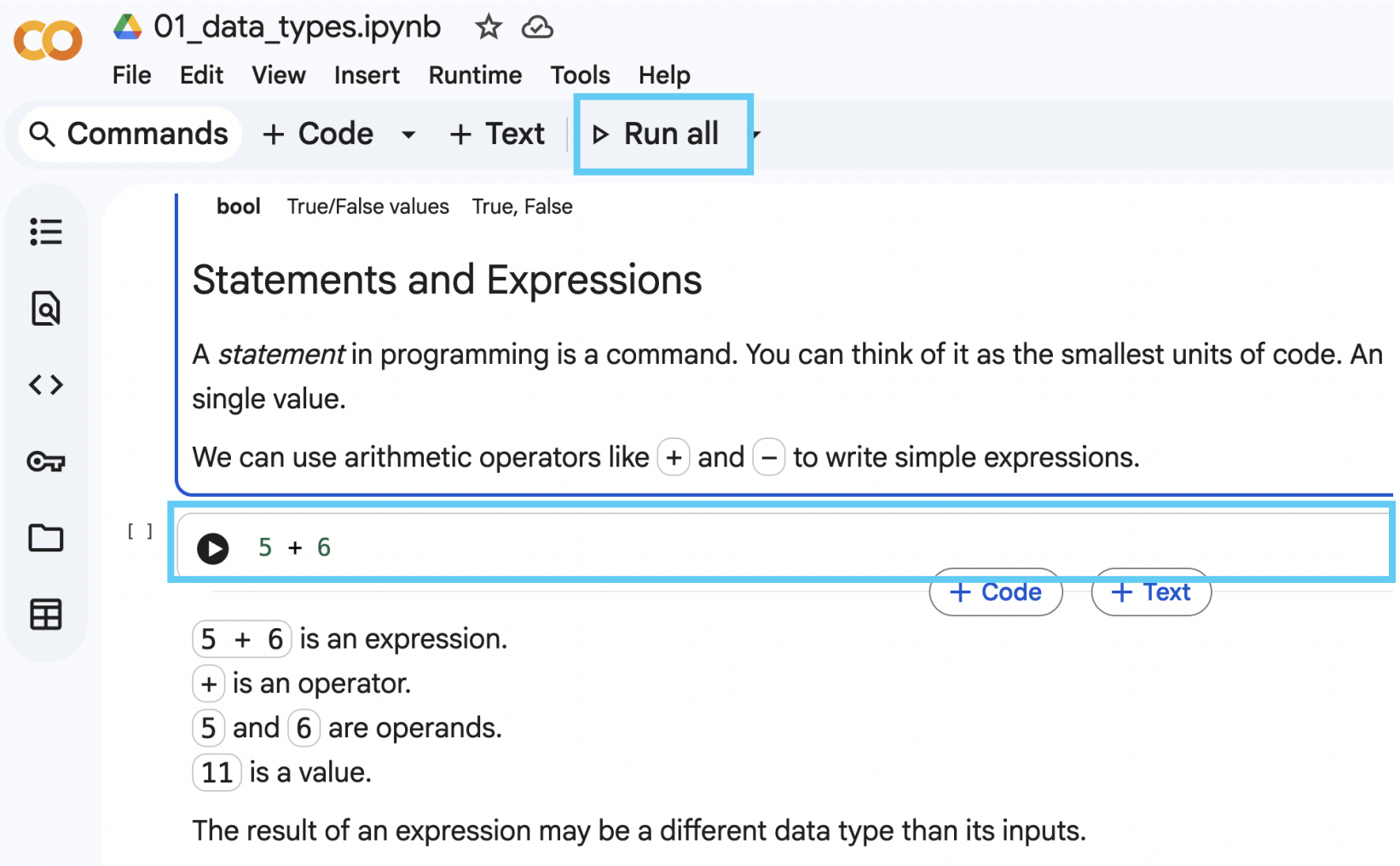
Jupyter notebook

- We will be using **Jupyter Notebooks**, which are files that end in **.ipynb**.

- **What is a Jupyter Notebook?**
  - A notebook lets you write **code and notes in the same place**
  - You can explain *what you're doing* while writing Python code
  - Everything stays together in one document!

- Jupyter Notebooks are made up of **cells**. Each cell can contain **either code or text.**
  - Code cells do the work
  - Text cells explain what the code is doing
  - Using both helps keep your notebook organized and easy to understand

Running Code



Code is written in **cells**

You can run:

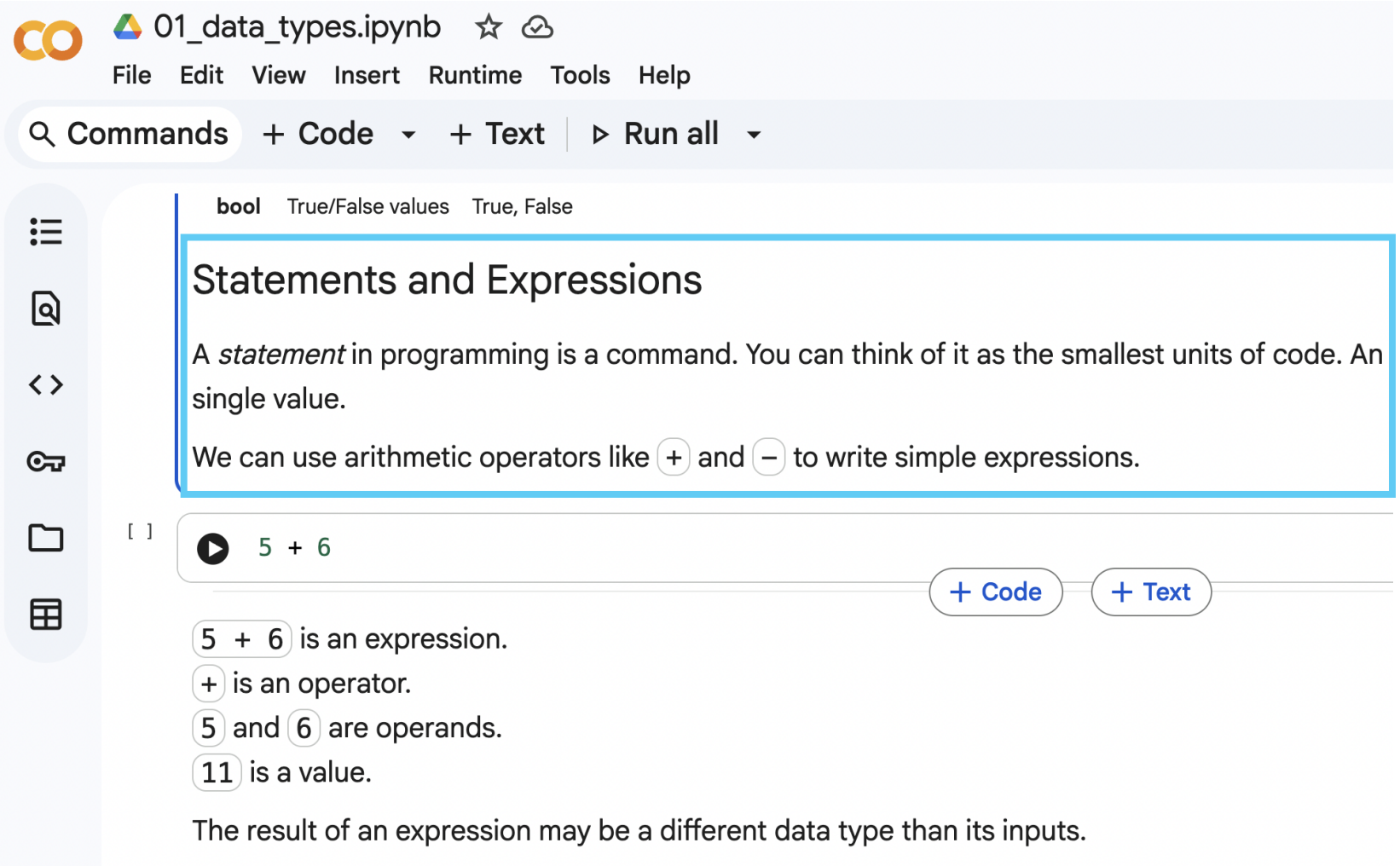
- One cell at a time (Click the ▶ button, or Press `Shift + Enter`

or

- All cells from top to bottom (Click Run all)

Results will appear right below the code

Writing Notes and Explanations



You can add **text cells** to write explanations, titles, or instructions

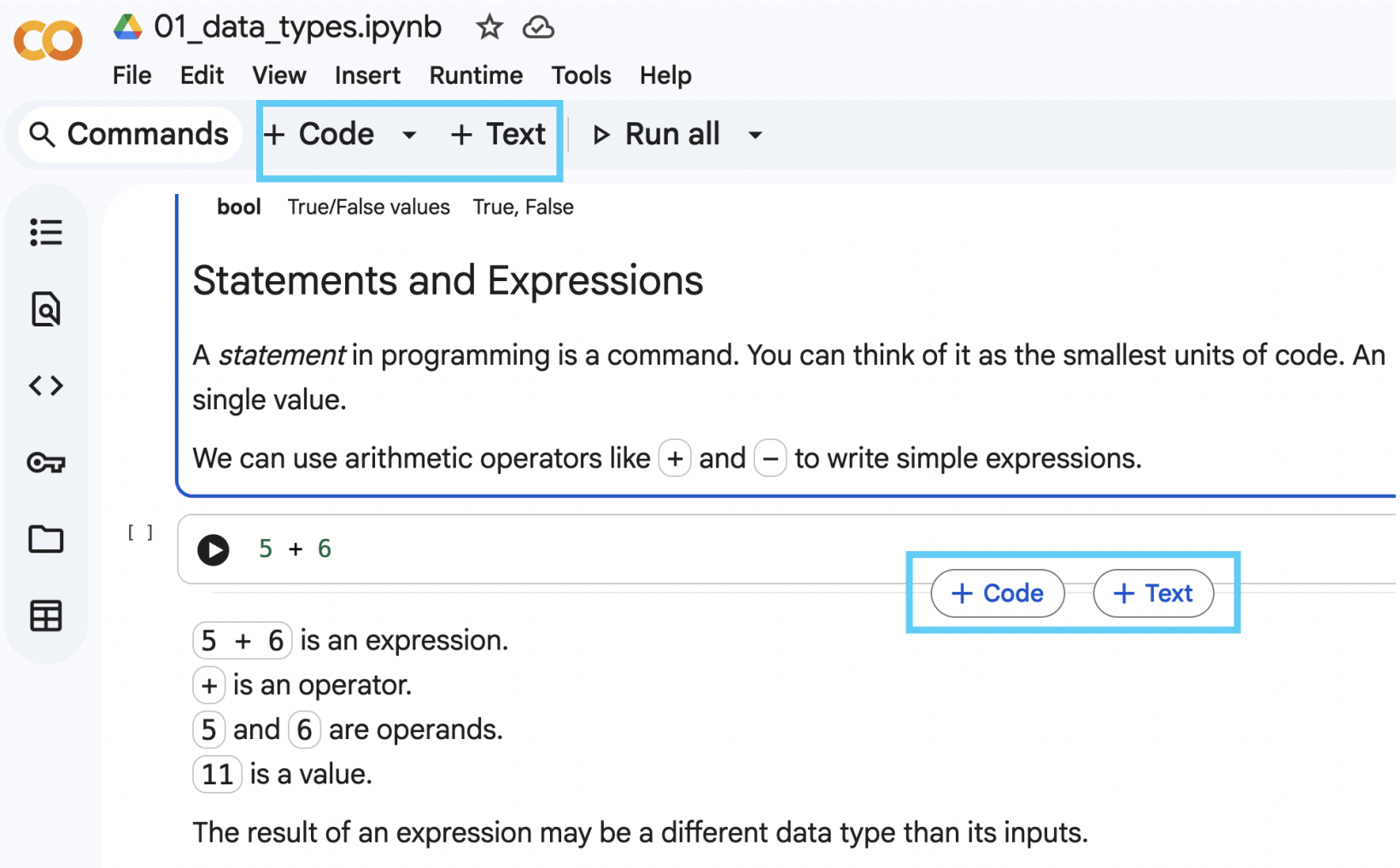
This helps you remember what your code does!

Text syntax

Syntax on left → Output on right

Plain text	Plain text
italics **bold**	*italics* **bold**
# Header 1 ## Header 2 … ###### Header 6	**Header 1** **Header 2** … **Header 6**

Adding New Code or Text



Click **+ Code** to add a new code cell

Click **+ Text** to add a new text (Markdown) cell

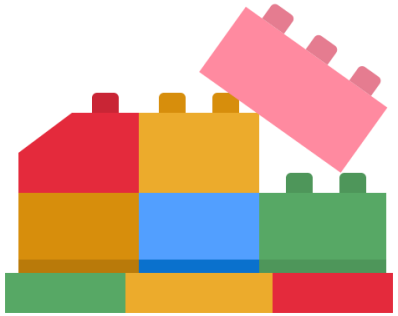
The new cell appears below the current cell

Data types

- Every value in Python has a data type, which tells the computer:
  - How the value is stored, and
  - What kinds of operations you can perform with it
- Here are a few basic types you will encounter:

Type	Description	Example
Integer number (or **Int**)	Whole number	10, -3, 42
Floating point number (or **Float**)	Decimal number	3.14, 0.01
String	Text strings	“Hello”, “Python”
Boolean (or **Bool**)	True/False values	True, False

Statements and expressions



- A **statement** is an instruction that **tells the computer to do something.**
  - You can think of statements as the basic building blocks of a program.
- An **expression** is a type of statement that combines operators and operands that **produces a value.**
- Example of an expression:

Operator: A symbol that tells Python to perform a specific action on values.

```
2 + 3
```

Operand: The value(s) that the operator acts on.

- When you run an expression, Python evaluates it and shows the result.

```
5
```

Arithmetic operators

Here are a few basic arithmetic operators you will encounter:

Operator	Description	Example	Result
+	Addition	`10 + 3`	`13`
-	Subtraction	`10 - 3`	`7`
*	Multiplication	`10 * 3`	`30`
/	Division	`10 / 3`	`3.333`
**	Exponentiation	`10**3`	`1000`
%	Modulo or remainder	`10 % 3`	`1`
//	Integer division (always rounds down)	`10 // 3`	`3`

Boolean operators

- Python has a Boolean data type: **Bool**.
  - Only two possible values: `True` or `False`
- **Comparison Expressions**
  - Use a comparison operators and produce a Boolean result (True or False)
  - Here are some common comparison operators:

Operator	Description	Example	Result
>	Greater than	`10 > 3`	`True`
>=	Greater than or equal to	`10 >= 3`	`True`
<	Less than	`10 < 3`	`False`
<=	Less than or equal to	`10 <= 3`	`False`
==	Equal to	`10 == 3`	`False`
!=	Not equal to	`10 != 3`	`True`

Order of operation

- Python follows a **specific order when evaluating expressions**. This is called operator precedence.

Order	Operator	Description
1	**	Exponentiation
2	-	Negation
3	*, /, //, %	Multiplication, division, integer division, and modulo
4	+, -	Addition and subtraction
5	<, <=, >, >=, ==, !=	Less than, less than or equal to, greater than, greater than or equal to, equal, not equal

- Examples

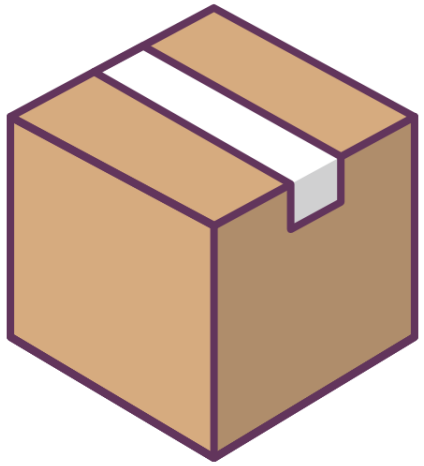
```
10 + 3 * 2
```

Multiplication first: 3*2 = 6 → 10 + 6 = 16

```
(10 + 3) * 2
```

Parentheses first: 10+3 = 13 → 13*2 = 26

Variables



- A variable is a name that stores a piece of information — like labeling a box so you remember what's inside.
- In Python:
  - Variable names can include letters, numbers, and underscores
  - Variable names cannot begin with a number
  - Variable names are case-sensitive (`age` and `Age` are different variables)
- Creating a Variable
  - Use the assignment operator =

```
degrees_celsius = 25
```

This is called an assignment statement!

Reassigning a variable

- You can use the value stored in one variable to calculate and create another variable.

```
degrees_fahrenheit = (9 / 5) * degrees_celsius + 32
```

- A **variable** stores a value, but you can **change the value** by **reassigning it**.
  - Reassigning a variable changes only that variable
  - Other variables that were assigned the old value do not change

```
degrees_celsius = 30
```

Reassigning degrees Celsius to 30 will **not change the value** of degrees Fahrenheit!

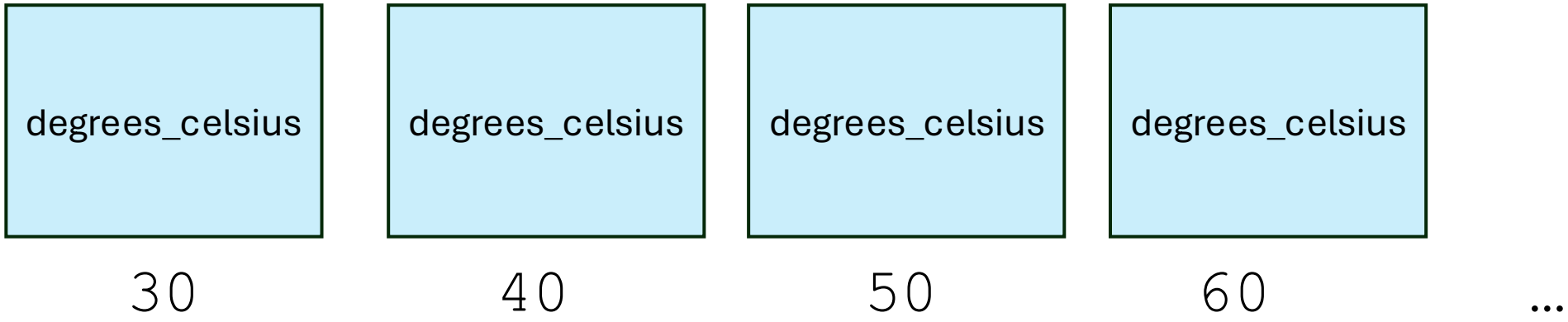
Using a Variable in Its Own Assignment

- You can update a variable based on its current value

```
degrees_celsius = degrees_celsius + 10
```

This would add 10 to the current value of degrees_celsius

- This is useful for running totals, counters, or updating values dynamically



Augmented assignment

- Instead of writing:

```
degrees_celsius = degrees_celsius + 10
```

- You can use a **shorthand operator** called an **augmented assignment**:

```
degrees_celsius += 10
```

This reads as, take the current value of degrees_celsius, add 10, and store the result back in degrees_celsius

- Similarly, subtraction, multiplication, division, exponentiation, integer division, and modulo can be used as an augmented assignment.

Exercises

- Try these on your own:
  - Create three variables:
    - `name`,
    - `age`,
    - `favorite_number`
  - Run the code cells to see your results!